Hmm, not sure yet.

Spencer Smith
University of Oklahoma
Norman, Oklahoma, United States
spencer.smith-1@ou.edu

Richard Veras
University of Oklahoma
Norman, Oklahoma, United States
richard.m.veras@ou.edu

Abstract

Sparse matrix-matrix multiplication (SpGEMM) is an important, performance-critical operation in many scientific and graph workloads, yet existing implementations rarely exploit fine-grained structural sparsity within the multiply itself, often focusing on a global compression strategy. In this work, we present a comprehensive benchmarking and instrumentation study of SpGEMM across a variety of storage formats, covering all permutations of input format pairings with dense output. Through runtime performance benchmarking and instrumentation, we observe the floating-point operation (FLOP) and memory operation (memop) counts for each format combination and derive analytical formulas that accurately approximate these costs as a function of matrix density and dimensions.

With these findings, we introduce a BLIS-inspired sparse matrix multiplication algorithm that preserves the hierarchical loop ordering and blocking of BLIS while replacing the innermost micro-kernel with heterogeneous format sparse blocks drawn from the aforementioned four formats. We formulate the selection of per-block formats as an integer non-linear optimization problem, using the derived cost formulas to identify the globally optimal format assignment for a given combination of input matrices and their respective density distribution over the micro-blocks. The result is a high performance SpGEMM that is adaptable to a wide range of sparse datasets.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Spencer Smith and Richard Veras. 2018. Hmm, not sure yet.. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

2 Background

3 Theory

4 Mechanism / Methodology

4.1 Instrumentation and Benchmarking

Each matrix multiplication kernel was initially derived using the TACO compiler as a reference, then rewritten by hand to incorporate a custom instrumentation layer. Each kernel is itself actually implemented as a C preprocessor macro, allowing the same implementation to be invoked both within the benchmarking framework and directly as a callable function in the sparse BLIS multiplication pipeline. The instrumentation layer is then built on top of this using a second set of macros, `LOAD`, `STORE`, and `FMADD`, which in their counting mode expand to the underlying operation plus a bit of bookkeeping to record memory and floating point operation activity. Each `FMADD` increments a floating point operation counter by two, while each `LOAD` and `STORE` increments both a memory operation counter and a byte counter. In their benchmarking mode the macros expand to only the base operations with no book-keeping overhead, allowing accurate runtime measurement. We observed the bookkeeping had a measurable and noticeable negative impact on performance.

Listing 1: Instrumentation macros

```
#ifdef OBSERVE_FLOPS
#define FMADD(dst, a, b) dst += (a * b);
    repetition_tester_count_flops(tester, 2)
#endif // OBSERVE_FLOPS
#ifdef OBSERVE_MEMOPS
#define LOAD(src) src;
    repetition_tester_count_memops(tester, 1);
    repetition_tester_count_bytes(tester,
    sizeof(src))
#define STORE(dst, src) dst = src;
    repetition_tester_count_memops(tester, 1);
    repetition_tester_count_bytes(tester,
    sizeof(dst))
#endif // OBSERVE_MEMOPS
```

Runtime performance is measured using a repetition testing framework inspired by Casey Muratori's Computer Enhance material (I probably oughta put a source for that

here). For each kernel, the framework repeatedly executes the multiplication over a fixed time window and records the minimum observed cycle count, using the x86 hardware timestamp counter, _RDTSC. Taking the minimum rather than the mean reduces the effect of OS scheduling noise as well as cold cache effects.

By sweeping input matrices across a range of densities, both the counters and the runtime measurements can be collected. Of course one at a time to eliminate any noise from the instrumentation on performance. We then obtain enough information for a direct comparison between predicted and observed operation counts as a function of sparsity. This sweep also provides the data required to plot a roofline model. The roofline performance bounds of peak floating point throughput and peak memory bandwidth were measured directly by running hand-written assembly loops designed to fully saturate the FMA and memory load ports of the target CPU. All results are visualized using a Python script built using Matplotlib.

The instrumentation macros actually ended up serving a dual purpose during development: beyond the data collection, their presence as visual markers in the kernel source made it straightforward to reason about which operations contributed to each term in the cost formula derivations described in the following subsection.

Listing 2: Implementation macro example, backslashes ommitted for formatting, STATEMENT expands to a do while()

```
#define csr_x_csr_impl STATEMENT(
  for (usize left_row = 0; left_row < left.
      row_count; left_row++)
  {
    usize left_row_start = LOAD(left.
        row_pointers[left_row]);
    usize left_row_end   = LOAD(left.
        row_pointers[left_row + 1]);

    for (usize i = left_row_start; i <
        left_row_end; i++)
    {
      usize left_col = LOAD(left.col_indices[i
          ]);
      f64 left_value = LOAD(left.values[i]);

      usize right_row_start = LOAD(right.
          row_pointers[left_col]);
      usize right_row_end   = LOAD(right.
          row_pointers[left_col + 1]);
      for (usize j = right_row_start; j <
          right_row_end; j++)
      {
        usize right_col = LOAD(right.
            col_indices[j]);
        f64 right_value = LOAD(right.values[j])
            ;

        usize output_index = left_row * output.
            col_count + right_col;

        f64 current_value = LOAD(output.values[
            output_index]);

        f64 result_value = current_value;
        FMADD(result_value, left_value,
            right_value);

        STORE(output.values[output_index],
            result_value);
      }
    }
  }
)
```

4.2 Formula Derivations

To enable format selection without runtime measurement, we derive analytical formulae for the floating point and memory operation counts of each kernel as a function of matrix dimensions and sparsity. For each kernel, the derivation follows a consistent procedure: we walk the loop nest from outermost to innermost, identify every `LOAD`, `STORE`, and `FMADD` site, and multiply each by the number of times its enclosing loop body executes. The instrumentation macros serve as explicit markers during this process, making each counted operation visually identifiable in the source code. We also note that we assume a uniform sparsity.

In some cases the loop bounds are not immediately obvious. For instance a loop iterating from a sparse row pointer start to end appears to iterate over a single row, but summing across all rows reveals the total iterations equal the non-zero count of that matrix. Recognising these collapsed bounds where nested loops together amount to a single pass over all non-zeros was an important step in several of the derivations.

We adopt a more descriptive subscript notation in place of the conventional M, N, K.

We define the following notation throughout:

- L_{RC}, L_{CC} — row and column count of the left matrix
- R_{CC} — column count of the right matrix (inner dimension is shared: $L_{CC} = R_{RC}$)
- L_{NZ}, R_{NZ} — non-zero counts of the left and right matrices
- matches $= L_{NZ} \cdot R_{NZ}/L_{CC}$ — expected number of multiply-accumulate operations under uniform sparsity with 2 sparse formats

Representative Derivations. We will present a few full derivations which we find representative of the whole process.

Dense × CSR. The outer loop iterates over L_{RC} rows of the left matrix. Within each row, the middle loop iterates over all L_{CC} rows of the right CSR matrix, reading two row pointers per iteration, $2 \cdot L_{RC} \cdot L_{CC}$ reads in total. However, across all iterations of this middle loop, the inner loop collectively visits every non-zero of the right matrix exactly once, so the total 2 inner iterations collapse to R_{NZ} per output row. This gives one left value load per non-zero per row, contributing another $L_{RC} \cdot L_{CC}$, and four reads/writes per

matched non-zero the right column index, right value, output load, and output store giving $4 \cdot L_{RC} \cdot R_{NZ}$.

$$\text{flops} = 2 \cdot L_{RC} \cdot R_{NZ} \tag{1}$$

$$\text{memops} = 3 \cdot L_{RC} \cdot L_{CC} + 4 \cdot L_{RC} \cdot R_{NZ} \tag{2}$$

COO $\times$ *COO.* The outer loop scans `left.row_indices` in full to group left entries into row segments, contributing L_{NZ} reads. Within each segment, the merge intersection advances `left.col_indices` through the segment, across all segments this totals another L_{NZ} reads. The right row index array resets to zero at the start of each left row segment and is fully scanned each time, costing $R_{NZ} \cdot L_S$ reads where L_S is the number of distinct left row segments. This is bounded above by both L_{RC}, the total number of rows, and L_{NZ}, since you cannot have more distinct rows than non-zeros, finally giving $L_S = \min(L_{RC}, L_{NZ})$. The min operation here helps to account for how this variable will change as density increases. At low densities L_{NZ} is very small and most non-zeroes exist on different rows, this flips as density increases and we approach a case where each row has at least one non-zero. Of course this holds true only for uniform sparsity. On a match, `left.values` is loaded once per matched left entry, giving L_{NZ} reads, and the right column index, right value, output load, and output store contribute $4 \cdot \text{matches}$. The expected match count assumes uniform sparsity, where each left non-zero at column k expects R_{NZ}/L_{CC} right non-zeros with row k, giving $\text{matches} = L_{NZ} \cdot R_{NZ}/L_{CC}$.

$$L_S = \min(L_{RC}, L_{NZ}) \tag{3}$$

$$\text{matches} = L_{NZ} \cdot R_{NZ}/L_{CC} \tag{4}$$

$$\text{flops} = 2 \cdot \text{matches} \tag{5}$$

$$\text{memops} = 3L_{NZ} + R_{NZ} \cdot L_S + 4 \cdot \text{matches} \tag{6}$$

Two format combinations, CSR×CSC and COO×CSC, present additional derivation difficulty compared to the rest. Both involve a merge intersection where both operands reset on different outer loop dimensions simultaneously: in CSR×CSC the left row entries reset per output column and the right column entries reset per output row, while in COO×CSC the same interaction occurs but compounded by the left row segment structure common in multiplies with COO. This seems to produce terms in the memory cost that are difficult, at least for us, to derive. The formulae given in Table 1 for these two cases capture the observed structure to the best of our abilities, though with considerably more error than the rest. All other fourteen combinations show strong agreement with observed counts across the full density range, as discussed in the Analysis section.

4.3 Format Selection Optimization

Given the cost formulae derived in the previous subsection, we formulate the problem of selecting an optimal storage format for each micro-block as an integer optimization program. Since in matrix multiplication one block may actually be multiplied against many other blocks. The objective is to minimise the total estimated cost across all block multiplications required to compute the full matrix product

Left	Right	Flops	Memops
DNS	DNS	$2L_{RC} \cdot L_{CC}R_{CC}$	$2L_{RC}L_{CC}R_{CC} + L_{RC}R_{CC}$
DNS	CSR	$2L_{RC} \cdot R_{NZ}$	$3L_{RC}L_{CC} + 4L_{RC}R_{NZ}$
DNS	CSC	$2L_{RC} \cdot R_{NZ}$	$3L_{RC}R_{CC} + 3L_{RC}R_{NZ}$
DNS	COO	$2L_{RC} \cdot R_{NZ}$	$2R_{NZ} + L_{RC}R_{NZ} + 4L_{RC}R_{NZ}$
CSR	DNS	$2L_{NZ} \cdot R_{CC}$	$2L_{RC} + 2L_{NZ} + 3L_{NZ}R_{CC}$
CSR	CSR	$2 \cdot \text{matches}$	$2L_{RC} + 4L_{NZ} + 4 \cdot \text{matches}$
CSR	CSC	$2 \cdot \text{matches}$	$2L_{RC} + 4L_{RC}R_{CC} + L_{NZ}R_{CC} + L_{RC}R$
CSR	COO	$2 \cdot \text{matches}$	$2L_{RC} + L_{NZ} + L_{RC}R_{NZ} + 4 \cdot \text{matches}$
CSC	DNS	$2L_{NZ} \cdot R_{CC}$	$2L_{CC} + 2L_{NZ} + 3L_{NZ}R_{CC}$
CSC	CSR	$2 \cdot \text{matches}$	$4L_{CC} + 2L_{NZ} + 4 \cdot \text{matches}$
CSC	CSC	$2 \cdot \text{matches}$	$2R_{CC} + 4R_{NZ} + 4 \cdot \text{matches}$
CSC	COO	$2 \cdot \text{matches}$	$2R_{NZ} + 2R_S + 4 \cdot \text{matches}$
COO	DNS	$2L_{NZ} \cdot R_{CC}$	$3L_{NZ} + 3L_{NZ}R_{CC}$
COO	CSR	$2 \cdot \text{matches}$	$5L_{NZ} + 4 \cdot \text{matches}$
COO	CSC	$2 \cdot \text{matches}$	$L_{NZ} + 2R_{CC}L_S + L_{NZ}R_{CC} + R_{NZ}L_S$
COO	COO	$2 \cdot \text{matches}$	$3L_{NZ} + R_{NZ}L_S + 4 \cdot \text{matches}$

Table 1: Arithmetic and memory operation count formulae for all sixteen format combinations. matches $= L_{NZ} \cdot R_{NZ}/L_{CC}$**,** $L_S = \min(L_{RC}, L_{NZ})$**,** $R_S = \min(L_{CC}, R_{NZ})$**.**

Each input matrix A and B is partitioned into micro-blocks of fixed size $L_{RC} \times L_{CC}$ and $L_{CC} \times R_{CC}$ respectively. The dimensions can also be viewed as those usually found at the M_r and N_r size found in BLIS. The density of each block is computed by simply counting its non-zeros, and put into one of our density buckets. A cost table is then computed for all combinations of left format, right format, and density bucket pair using the above derived formulae, treating flops and memory operations as a combined cost weighted by an assumed memory access penalty.

For each block of A at position (i, p) and each block of B at position (p, j), we introduce binary indicator variables $a_{i,p,f}$ and $b_{p,j,f}$ denoting whether block (i, p) of A or block (p, j) of B is assigned format $f \in \{\text{dense, CSR, CSC, COO}\}$. A single-assignment constraint ensures each block is assigned exactly one format:

$$\sum_f a_{i,p,f} = 1 \quad \forall\, i, p \tag{7}$$

$$\sum_f b_{p,j,f} = 1 \quad \forall\, p, j \tag{8}$$

The objective minimises the total estimated cost over all block multiplications:

$$C = \min \sum_{i,j,p} \sum_{f_A, f_B} \text{cost}[f_A, f_B, d^A_{i,p}, d^B_{p,j}] \cdot a_{i,p,f_A} \cdot b_{p,j,f_B} \tag{9}$$

The product $a_{i,p,f_A} \cdot b_{p,j,f_B}$ makes this objective 'bilinear' (is this right? from looking stuff up it seems like this is the right term for it...) in the decision variables, which constitutes a Mixed Integer Nonlinear Program (MINLP). The solver is implemented using Pyomo (probably ought to cite that here)

with MindtPy as the MINLP solver backend, using GLPK for the MIP subproblems and IPOPT for the NLP solver. Once solved, the optimal format assignments are written alongside the input matrices to a binary file consumed by the sparse BLIS multiplication program.

- $B_{row} = \texttt{MATRIX\_SIZE}/L_{RC}$ — number of block rows in A/'Left'
- $B_{col} = \texttt{MATRIX\_SIZE}/R_{CC}$ — number of block columns in B/'Right'
- $B_{inner} = \texttt{MATRIX\_SIZE}/L_{CC}$ — number of blocks along the shared inner dimension

So far, the primary limitation of this approach is solver scalability. The number of decision variables grows as $O(B_{row} \cdot B_{inner} \cdot B_{col} \cdot |Formats|)$ where $|Formats| = 4$ is the number of formats, and the bilinear objective compounds this as problem size increases. In practice the solver becomes intractable for matrix sizes beyond approximately 512×512 with the micro-block sizes used in our experiments. One direction may be increasing the micro-block size to alleviate the rapid growth of the number of decision variables.

4.4 Sparse with BLIS structure

The sparse BLIS implementation follows the hierarchical loop structure of BLIS (again, probably ought to cite that paper here), preserving its cache-level blocking parameters m_c, n_c, k_c and micro-kernel blocking parameters m_r, n_r, k_u as compile-time constants. Rather than operating on scalar elements or fixed-format compressed matrices, the innermost micro-kernel operates on heterogeneous sparse blocks — each of size $m_r \times k_u$ for the left matrix and $k_u \times n_r$ for the right — where each block may independently be stored in any of the four formats.

Prior to multiplication, each input matrix is packed into a `Multisparse_Matrix` structure by the `multi_sparsify` function, which partitions the dense input into blocks and converts each block to its assigned format according to the format assignment produced by the ILP solver, in the order consumed by the multiply. This packing is performed once and before the actual operation begins, as opposed to the standard BLIS practice where packing into the A_c and B_c panels happens during operation.

Listing 3: Multisparse Matrix struct

```
struct Matrix_Union
{
  Matrix_Format format;
  union
  {
    Dense_Matrix dense;
    CSR_Matrix   csr;
    CSC_Matrix   csc;
    COO_Matrix   coo;
  };
};
struct Multisparse_Matrix
{
  // Overall.
  usize row_count;
  usize col_count;

  Matrix_Union *blocks;
  usize blocks_row_count;
  usize blocks_col_count;
};
```

The multiplication itself is structured as the standard five-loop BLIS nest, but now iterating over block indices rather than element indices. At the innermost level, each pair of left and right blocks is dispatched to the appropriate kernel via a computed enum value encoding the format combination, covering all sixteen format pairings. The result of each micro-kernel is accumulated into a small dense temporary buffer of size $m_r \times n_r$ before being written back to the output matrix, very similarly to the original BLIS.

Three configurations are benchmarked against each other: the solved optimal heterogeneous format assignment, a uniform block format baseline where all micro-blocks use the same user-specified format, and a global compression baseline where the entire matrix is compressed into a single format without any type of block structure. Correctness of the sparse BLIS output is verified by comparison against a reference multiplication within an epsilon.

4.5 The Kron Stuff

5 Analysis

5.1 Format Benchmarking and Instrumentation Results

For the graphs below, we sweeped both left and right matrices through a range of densities.

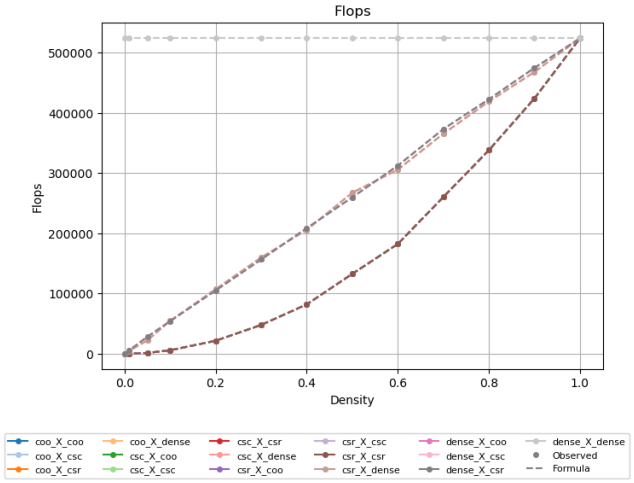

Figure 1: Observed and Formula FLOPs.

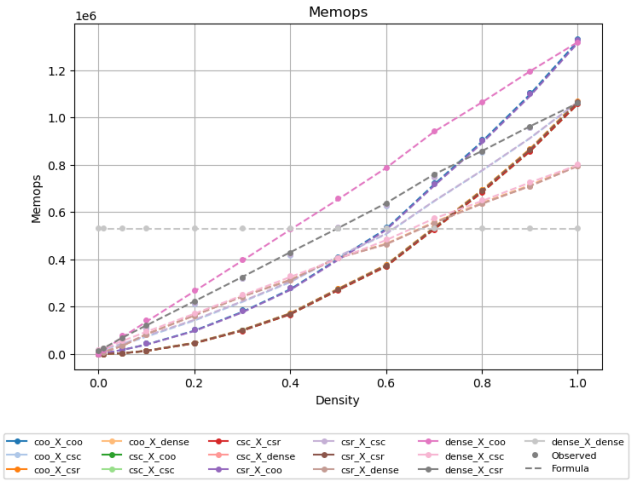

Figure 2: Observed and Formula memops.

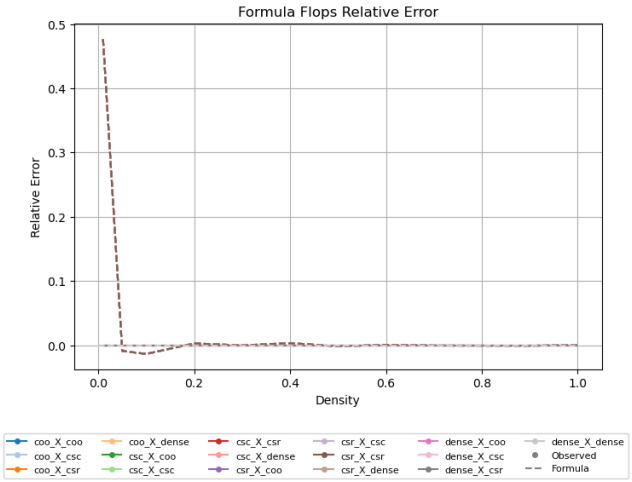

Figure 3: FLOPs: Relative error of Formula vs Observed.

5.2 Format Selection Optimization Results

5.3 Sparse BLIS Results

5.4 The Kron Stuff Results

6 Summary

7 Acknowledgments

Identification of funding sources and other support, and thanks to individuals and groups that assisted in the research and the preparation of the work should be included in an acknowledgment section, which is placed just before the reference section in your document.

This section has a special environment:

```
\begin{acks}
...
\end{acks}
```

so that the information contained therein can be more easily collected during the article metadata extraction phase, and to ensure consistency in the spelling of the section heading.

Authors should not prepare this section as a numbered or unnumbered `\section`; please use the "`acks`" environment.

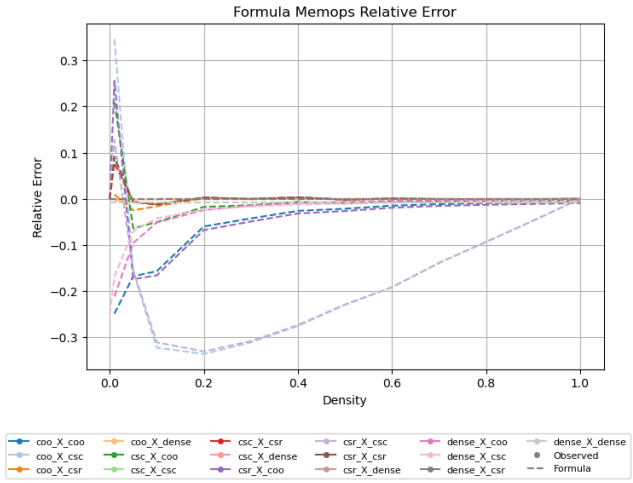

Figure 4: memops: Relative error of Formula vs Observed.

8 Appendices

If your work needs an appendix, add it before the "`\end{document}`" command at the conclusion of your source document.

Start the appendix with the "`appendix`" command:

```
\appendix
```

and note that in the appendix, sections are lettered, not numbered. This document has two appendices, demonstrating the section and subsection identification method.

Received 30 April 2026